Logistic Regression

Hotel Booking Demand | Data Science Toolbox

Logistic Regression

We use logistic regression to predict whether a hotel booking will be cancelled, using the group's cleaned dataset and agreed train/validation/test split. We assess performance using the Brier score and compare our final model with Random Forest and XGBoost.

1. Why logistic regression?

We chose logistic regression as an interpretable baseline for predicting cancellations. Unlike ordinary linear regression, it is designed for a binary outcome and gives predictions between zero and one, which we can interpret as probabilities.

Our EDA showed that cancellation rates depend on several booking characteristics, particularly lead time, hotel type and deposit policy. Logistic regression lets us examine these relationships explicitly and assess whether adding further predictors improves performance.

The main limitation is that it assumes a linear relationship with the log-odds of cancellation unless we specify otherwise. We therefore start with a simple model and test whether transformations, splines and interactions improve its predictions.

2. How logistic regression works

2.1 The logistic model

Let $Y_i = 1$ if booking i is cancelled, and $Y_i = 0$ otherwise. We model the outcome as

$$Y_i \mid \mathbf{x}_i \sim \text{Bernoulli}(p_i),$$

where p_i is the probability of cancellation given the booking's predictors.

We define the linear predictor as

$$\eta_i = \beta_0 + \sum_{j=1}^{k} \beta_j \, x_{ij}.$$

The logit link relates this to the cancellation probability:

$$\log\left(\frac{p_i}{1 - p_i}\right) = \eta_i.$$

Rearranging gives

$$p_i = \frac{1}{1 + e^{-\eta_i}}.$$

This ensures that our predictions lie between zero and one. For a linear predictor, e^{β_j} gives the multiplier on the odds of cancellation for a one-unit increase in x_j, holding the other terms fixed. For categorical variables, coefficients are relative to a reference category.

2.2 Maximum likelihood estimation

We estimate the coefficients by maximising the Bernoulli likelihood:

$$L(\boldsymbol{\beta}) = \prod_{i=1}^{n} p_i^{y_i} (1 - p_i)^{1-y_i}.$$

Taking logarithms gives

$$\ell(\boldsymbol{\beta}) = \sum_{i=1}^{n} [y_i \log p_i + (1 - y_i) \log(1 - p_i)].$$

Writing X for the design matrix and $\mathbf{p}$ for the vector of predicted probabilities, the gradient and Hessian are

$$\nabla \ell = X^\top (\mathbf{y} - \mathbf{p}), \qquad \nabla^2 \ell = -X^\top W X,$$

where $W = \operatorname{diag}\big(p_i(1 - p_i)\big)$.

These give the Newton–Raphson/Fisher-scoring update

$$\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} + (X^\top W X)^{-1} X^\top (\mathbf{y} - \mathbf{p}).$$

R's `glm()` function fits logistic regression using iteratively reweighted least squares (IRLS), which implements this approach through numerical linear algebra rather than explicitly calculating the inverse.

We compare the models mainly using the group's chosen metric, the **Brier score**:

$$BS = \frac{1}{n} \sum_{i=1}^{n} (\hat{p}_i - y_i)^2.$$

This measures the mean squared error of our predicted probabilities, with smaller values indicating better performance. The group's separate performance-metrics report explains the choice of this metric in more detail.

2.3 Non-linear effects and interactions

A basic logistic regression assumes each numerical predictor has a linear effect on the log-odds. Since this may not be realistic for lead time or room price, we also investigate transformations and natural cubic splines.

Using spline basis functions B_r, we can represent a non-linear relationship as

$$f(z_i) = \sum_{r=1}^{q} \gamma_r B_r(z_i).$$

We fit these using `splines::ns()` in R, allowing a smooth relationship without assuming a single straight line.

We also test interactions, particularly between hotel and ADR, to allow the effect of room price to differ between hotels. Finally, we investigate grouping numerical count variables to avoid assuming that every additional special request or previous cancellation has the same effect.

3. Preparing the data

We use the cleaned dataset and train/validation/test split established in the group's preprocessing notebook, rather than repeating that work here. The exact arrival date and year are excluded as predictors, while arrival month is retained to capture possible seasonality.

All model development uses the training and validation data. We leave the test set for the final comparison with Random Forest and XGBoost.

```
library(readr)

## Warning: package 'readr' was built under R version 4.5.3

library(dplyr)

##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union

library(ggplot2)
library(splines)
library(knitr)

data <- read_csv("~/Data Science Toolbox/sprint1/hotel_bookings_clean.csv")

## Rows: 118558 Columns: 30

## ── Column specification
──────────────────────────────────────────────────────
```

```
## Delimiter: ","
## chr  (11): hotel, arrival_date_month, meal, country, market_segment,
distrib...
## dbl  (18): is_canceled, lead_time, arrival_date_year,
arrival_date_week_numb...
## date  (1): arrival_date
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this
message.
```

```
train <- subset(data, split == "train")
validation <- subset(data, split == "validation")
test <- subset(data, split == "test")

split_summary <- data %>%
  count(split, name = "bookings")

kable(split_summary, caption = "Common dataset split used by all three
models")
```

Common dataset split used by all three models

split	bookings
test	17903
train	83324
validation	17331

The shared dataset contains 83,324 training, 17,331 validation and 17,903 test bookings. Identical records were kept within the same split, and the test outcomes were reserved for the final comparison.

4. Model development and selection

4.1 A simple baseline

We started with just `hotel` and `lead_time`, as the EDA suggested both were associated with cancellations. This also gave us a simple baseline to compare more complicated models against.

```
model1 <- glm(
  is_canceled ~ hotel + lead_time,
  data = train,
  family = binomial(link = "logit")
)

round(coef(model1), 4)
```

```
##       (Intercept) hotelResort Hotel         lead_time
##           -0.9409           -0.5796            0.0055
```

As expected, longer lead times increased the estimated odds of cancellation, while Resort Hotel had lower odds than City Hotel.

After looking at other analyses of this dataset, including Chakraborty (2020), we tried transforming lead time using `log1p(lead_time)`. This seemed appropriate given the non-linear pattern in the EDA, and also works for bookings with zero lead time.

```
model2 <- glm(
  is_canceled ~ hotel + log1p(lead_time),
  data = train,
  family = binomial(link = "logit")
)

p1 <- predict(model1, validation, type = "response")
p2 <- predict(model2, validation, type = "response")

brier1 <- mean((p1 - validation$is_canceled)^2)
brier2 <- mean((p2 - validation$is_canceled)^2)
```

The transformation reduced the validation Brier score from 0.21062 to 0.20596 without adding any parameters, so we kept it for the following models.

4.2 Adding more predictors

We next added booking characteristics identified during the EDA. Model 3 included deposit type, market segment, customer type, booking history, special requests, ADR and arrival month.

```
model3 <- glm(
  is_canceled ~ hotel +
    log1p(lead_time) +
    deposit_type +
    market_segment +
    customer_type +
    adr +
    previous_cancellations +
    is_repeated_guest +
    total_of_special_requests +
    arrival_date_month,
  data = train,
  family = binomial(link = "logit")
)

p3 <- predict(model3, validation, type = "response")
brier3 <- mean((p3 - validation$is_canceled)^2)
```

We then added `country` in Model 4, as cancellation rates differed substantially between countries in the EDA.

```
model4 <- update(model3, . ~ . + country)

p4 <- predict(model4, validation, type = "response")
brier4 <- mean((p4 - validation$is_canceled)^2)
```

Finally, Model 5 included the remaining guest, stay and booking characteristics to see whether they provided any additional predictive information.

```
model5 <- update(
  model4,
  . ~ . +
    previous_bookings_not_canceled +
    stays_in_week_nights +
    stays_in_weekend_nights +
    adults +
    children +
    babies +
    meal +
    reserved_room_type +
    distribution_channel +
    has_company
)

p5 <- predict(model5, validation, type = "response")
brier5 <- mean((p5 - validation$is_canceled)^2)
```

Comparing the models

```
models <- list(model1, model2, model3, model4, model5)

comparison15 <- data.frame(
  Model = c(
    "1: Simple baseline",
    "2: Log-transformed lead time",
    "3: Initial expanded model",
    "4: Adding country",
    "5: Additional booking information"
  ),
  Parameters = sapply(models, function(m) m$rank),
  AIC = sapply(models, AIC),
  Validation_Brier = c(
    brier1, brier2, brier3, brier4, brier5
  )
)

knitr::kable(
  comparison15,
```

```
  digits = c(0, 0, 2, 5),
  caption = "Comparison of initial logistic regression models"
)
```

Comparison of initial logistic regression models

Model	Parameters	AIC	Validation_Brier
1: Simple baseline	3	102167.25	0.21062
2: Log-transformed lead time	3	99921.96	0.20596
3: Initial expanded model	29	75386.27	0.14989
4: Adding country	68	67314.71	0.13468
5: Additional booking information	89	66263.87	0.13179

Adding predictors substantially improved performance, reducing the Brier score from 0.20596 in Model 2 to 0.13179 in Model 5. The largest improvements came from the initial expanded model and adding country, with smaller gains from the remaining booking information.

Model 5 became our starting point for further refinement. Some expanded models produced warnings about fitted probabilities being numerically close to zero or one. Our exploratory checks found no complete or quasi-complete separation in Model 5, with the extreme predictions partly explained by the unusually high cancellation rate among non-refundable bookings.

4.3 Ridge and LASSO regression

Model 5 had 88 coefficients excluding the intercept, so we investigated whether regularisation could improve predictions or simplify the model.

We compared **ridge and LASSO regression**, which penalise large coefficients when fitting the model. The penalised objective is

$$-\frac{1}{n}\ell(\boldsymbol{\beta}) + \lambda\left[\frac{1-\alpha}{2}\sum_{j=1}^{p}\beta_j^2 + \alpha\sum_{j=1}^{p}\left|\beta_j\right|\right],$$

where λ controls the penalty strength. Ridge ($\alpha = 0$) shrinks coefficients towards zero, while LASSO ($\alpha = 1$) can set them exactly to zero, effectively removing predictors.

We fitted both using `glmnet`, selecting λ through five-fold cross-validation on the training data. The same folds were used for both methods.

```
regularisation_results <- data.frame(
  Model = c(
    "Model 5: Ordinary logistic",
    "Ridge",
    "LASSO (minimum CV error)",
```

```
    "LASSO (1SE rule)"
  ),
  Nonzero_Coefficients = c(88, 88, 88, 83),
  Validation_Brier = c(
    0.1317853,
    0.1360217,
    0.1318867,
    0.1321741
  )
)

knitr::kable(
  regularisation_results,
  digits = 5,
  caption = "Validation performance of regularised regression"
)
```

Validation performance of regularised regression

Model	Nonzero_Coefficients	Validation_Brier
Model 5: Ordinary logistic	88	0.13179
Ridge	88	0.13602
LASSO (minimum CV error)	88	0.13189
LASSO (1SE rule)	83	0.13217

Neither method improved on ordinary logistic regression. Ridge performed worse, while LASSO gave almost the same Brier score but retained all 88 coefficients at its optimal penalty. Using the stronger one-standard-error penalty reduced this to 83 coefficients, with slightly worse predictions.

These were exploratory comparisons using randomly assigned folds, rather than the duplicate-grouped folds used later. Since neither method offered a clear advantage for Model 5, we continued with ordinary logistic regression.

4.4 Allowing for non-linear relationships

Although the log transformation improved lead time, we were still assuming a linear relationship with the log-odds of cancellation after transformation. We therefore tried **natural cubic splines**, which allow the relationship to change smoothly without fitting a single straight line.

Natural cubic splines join piecewise cubic functions at knots, with linear behaviour beyond the boundary knots. We used `splines::ns()` with four degrees of freedom, as introduced in Section 2.3.

Model 6: A spline for lead time

We replaced the log-transformed lead-time term in Model 5 with a natural cubic spline.

```r
model6 <- update(
  model5,
  . ~ . - log1p(lead_time) +
    splines::ns(log1p(lead_time), df = 4)
)

p6 <- predict(model6, validation, type = "response")
brier6 <- mean((p6 - validation$is_canceled)^2)
```

The validation Brier score fell from 0.13179 to 0.13108, suggesting that allowing a more flexible lead-time relationship improved our predictions.

Visualising the spline

We compared Models 5 and 6 by varying lead time for an example City Hotel booking while keeping its other characteristics fixed.

```r
# Select an example booking
example_row <- which(
  train$hotel == "City Hotel" &
  train$deposit_type == "No Deposit" &
  train$previous_cancellations == 0
)[1]

example_booking <- train[example_row, , drop = FALSE]

# Vary lead time while keeping other predictors fixed
lead_grid <- seq(0, 365, by = 5)

new_data <- example_booking[
  rep(1, length(lead_grid)), ,
  drop = FALSE
]

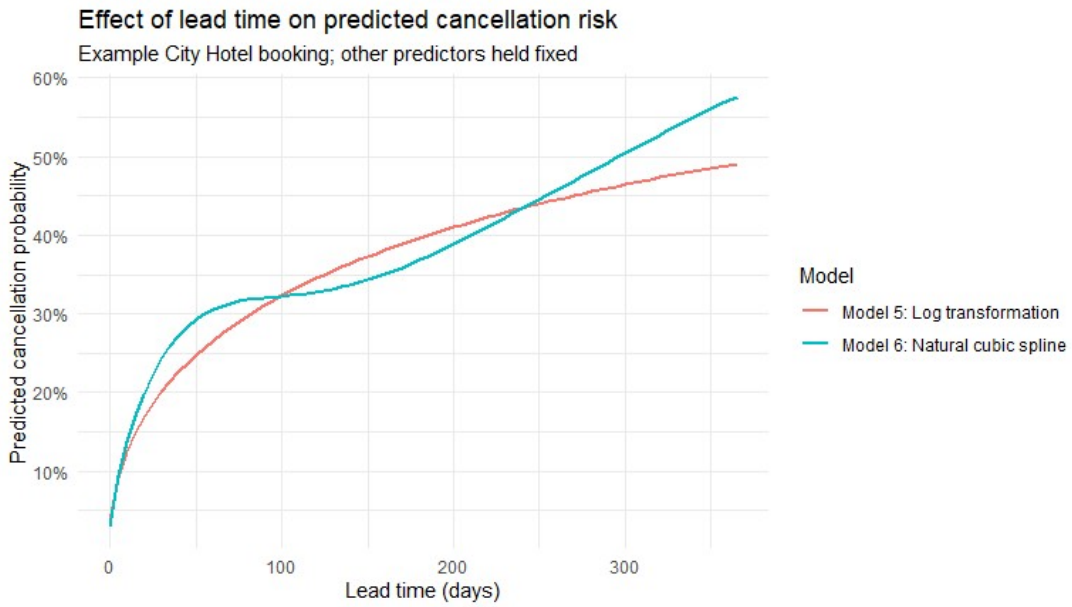
new_data$lead_time <- lead_grid

lead_plot <- data.frame(
  Lead_Time = lead_grid,
  Model_5 = as.numeric(
    predict(model5, new_data, type = "response")
  ),
  Model_6 = as.numeric(
    predict(model6, new_data, type = "response")
  )
)

lead_plot_long <- tidyr::pivot_longer(
  lead_plot,
  cols = c(Model_5, Model_6),
```

```
  names_to = "Model",
  values_to = "Probability"
)

ggplot(
  lead_plot_long,
  aes(x = Lead_Time, y = Probability, colour = Model)
) +
  geom_line(linewidth = 1) +
  scale_colour_discrete(
    labels = c(
      "Model 5: Log transformation",
      "Model 6: Natural cubic spline"
    )
  ) +
  scale_y_continuous(labels = scales::percent) +
  labs(
    title = "Effect of lead time on predicted cancellation risk",
    subtitle = "Example City Hotel booking; other predictors held fixed",
    x = "Lead time (days)",
    y = "Predicted cancellation probability",
    colour = "Model"
  ) +
  theme_minimal()
```



The spline allows the estimated cancellation risk to change more flexibly across different lead times, rather than following the fixed relationship imposed by Model 5. The plot shows predictions for one example booking, not average cancellation rates across all bookings.

Model 7: A spline for ADR

We next replaced the linear ADR term with a natural cubic spline, allowing the relationship between room price and cancellation risk to vary across price ranges.

```
model7 <- update(
  model6,
  . ~ . - adr +
    splines::ns(adr, df = 4)
)

p7 <- predict(model7, validation, type = "response")
brier7 <- mean((p7 - validation$is_canceled)^2)
```

Model 8: Transforming previous successful bookings

We also tried a log transformation of previous_bookings_not_canceled, since most guests had no previous bookings and relatively few had large counts.

```
model8 <- update(
  model7,
  . ~ . - previous_bookings_not_canceled +
    log1p(previous_bookings_not_canceled)
)

p8 <- predict(model8, validation, type = "response")
brier8 <- mean((p8 - validation$is_canceled)^2)
```

Comparing the non-linear models

```
nonlinear_results <- data.frame(
  Model = c(
    "Model 5: Log lead time",
    "Model 6: Lead-time spline",
    "Model 7: Lead-time and ADR splines",
    "Model 8: Log previous bookings"
  ),
  Parameters = c(
    model5$rank,
    model6$rank,
    model7$rank,
    model8$rank
  ),
  AIC = c(
    AIC(model5),
    AIC(model6),
    AIC(model7),
    AIC(model8)
  ),
  Validation_Brier = c(
    brier5,
    brier6,
```

```
    brier7,
    brier8
  )
)

knitr::kable(
  nonlinear_results,
  digits = c(0, 0, 2, 5),
  caption = "Comparison of non-linear logistic regression models"
)
```

Comparison of non-linear logistic regression models

Model	Parameters	AIC	Validation_Brier
Model 5: Log lead time	89	66263.87	0.13179
Model 6: Lead-time spline	92	66046.31	0.13108
Model 7: Lead-time and ADR splines	95	65965.97	0.13098
Model 8: Log previous bookings	95	65887.17	0.13102

Both splines improved performance, with Model 7 achieving a validation Brier score of 0.13098. Model 8 had a lower AIC but a slightly worse Brier score (0.13102), so we did not keep the previous-bookings transformation.

We therefore continued with Model 7, retaining the lead-time and ADR splines.

4.5 Interactions between predictors

So far, we have allowed non-linear relationships with lead time and ADR, but assumed these relationships are the same on the log-odds scale for both hotels. We therefore tested interactions between `hotel` and the two splines.

For example, a hotel–ADR interaction can be written as

$$\text{logit}(p_i) = \beta_0 + \beta_h H_i + \sum_{j=1}^{4} \gamma_j \, B_j(a_i) + H_i \sum_{j=1}^{4} \delta_j \, B_j(a_i) + \boldsymbol{\beta}^T \mathbf{z}_i,$$

where $H_i = 1$ for Resort Hotel and zero for City Hotel, a_i is ADR, and B_j are the spline basis functions. The interaction coefficients δ_j allow the ADR relationship to take a different shape for each hotel.

Model 9: Hotel and lead time

We first added an interaction between hotel and the lead-time spline.

```
model9 <- update(
  model7,
  . ~ . +
    hotel:splines::ns(log1p(lead_time), df = 4)
```

```
)

p9 <- predict(model9, validation, type = "response")
brier9 <- mean((p9 - validation$is_canceled)^2)

# Likelihood-ratio test
lr9 <- anova(model7, model9, test = "Chisq")
```

The likelihood-ratio test was highly significant ($p = 3.31 \times 10^{-8}$), but the validation Brier score barely improved. Although the interaction improved training fit, it offered little predictive benefit.

Model 10: Hotel and ADR

We then tested an interaction between hotel and ADR, again starting from Model 7.

```
model10 <- update(
  model7,
  . ~ . +
    hotel:splines::ns(adr, df = 4)
)

p10 <- predict(model10, validation, type = "response")
brier10 <- mean((p10 - validation$is_canceled)^2)

# Likelihood-ratio test
lr10 <- anova(model7, model10, test = "Chisq")
```

This interaction was also significant ($p = 1.36 \times 10^{-11}$) and gave a more noticeable improvement in validation performance.

Comparing the interaction models

```
interaction_results <- data.frame(
  Model = c(
    "Model 7: No interactions",
    "Model 9: Hotel x lead time",
    "Model 10: Hotel x ADR"
  ),
  Parameters = c(
    model7$rank,
    model9$rank,
    model10$rank
  ),
  AIC = c(
    AIC(model7),
    AIC(model9),
    AIC(model10)
  ),
  Validation_Brier = c(
```

```
    brier7,
    brier9,
    brier10
  )
)

knitr::kable(
  interaction_results,
  digits = c(0, 0, 2, 6),
  caption = "Comparison of hotel interaction models"
)
```

Comparison of hotel interaction models

Model	Parameters	AIC	Validation_Brier
Model 7: No interactions	95	65965.97	0.130980
Model 9: Hotel x lead time	99	65933.41	0.130978
Model 10: Hotel x ADR	99	65917.17	0.130441

Both interactions improved the training fit, but only the hotel–ADR interaction noticeably improved validation predictions, reducing the Brier score from 0.13098 to 0.13044.

With over 83,000 training bookings, even small differences can be statistically significant without being particularly useful for prediction. We therefore carried forward **Model 10**, which includes the hotel–ADR interaction but not the lead-time interaction.

4.6 Adding booking agent information

We next investigated `agent`, which identifies the travel agency associated with each booking. We had initially excluded it because it contains 86 categories, but cancellation rates varied considerably between agents.

```
agent_summary <- train %>%
  group_by(agent) %>%
  summarise(
    Bookings = n(),
    Cancellation_Rate = round(100 * mean(is_canceled), 1),
    .groups = "drop"
  ) %>%
  arrange(desc(Bookings)) %>%
  slice_head(n = 8)

knitr::kable(
  agent_summary,
  col.names = c("Agent", "Bookings", "Cancellation rate (%)"),
  caption = "Cancellation rates for the eight most common agents"
)
```

Cancellation rates for the eight most common agents

Agent	Bookings	Cancellation rate (%)
9	22411	41.5
none	11224	25.0
240	9616	39.9
1	5173	73.6
Other	3553	31.0
14	2515	17.0
7	2448	13.7
6	2246	29.4

For example, Agent 1 had a cancellation rate of 73.6%, compared with 13.7% for Agent 7. These differences could reflect the types of customers and bookings handled by each agency, but suggested that agent information might improve predictions.

Model 11: Adding agent

We added agent as a categorical predictor to Model 10.

```
model11 <- update(model10, . ~ . + agent)

stopifnot(model11$converged)

p11 <- predict(model11, validation, type = "response")
brier11 <- mean((p11 - validation$is_canceled)^2)

agent_results <- data.frame(
  Model = c("Model 10", "Model 11: Adding agent"),
  Parameters = c(model10$rank, model11$rank),
  AIC = c(AIC(model10), AIC(model11)),
  Validation_Brier = c(brier10, brier11)
)

knitr::kable(
  agent_results,
  digits = c(0, 0, 2, 5),
  caption = "Effect of adding booking agent information"
)
```

Effect of adding booking agent information

Model	Parameters	AIC	Validation_Brier
Model 10	99	65917.17	0.13044
Model 11: Adding agent	184	62557.10	0.12609

Adding agent reduced the validation Brier score from 0.13044 to 0.12609, although it increased the number of parameters from 99 to 184. We kept Model 11 as a candidate, but the extra complexity raised concerns about overfitting. Predictions for previously unseen agents would also require a way of handling new categories. We therefore assessed its performance further using cross-validation.

4.7 Grouping special requests and previous cancellations

We also reconsidered `total_of_special_requests` and `previous_cancellations`, which had been treated as linear predictors. Both were heavily concentrated at zero, so assuming each additional request or cancellation had the same effect on log-odds seemed restrictive.

We grouped special requests into 0, 1, 2 and 3+, and previous cancellations into 0, 1 and 2+. This allows different effects for each category while combining the rarer higher counts.

```
# Apply the same transformations to training and validation
add_groups <- function(df) {
  df %>%
    mutate(
      requests_group = factor(
        pmin(total_of_special_requests, 3),
        levels = 0:3,
        labels = c("0", "1", "2", "3+")
      ),
      prev_cancel_group = factor(
        pmin(previous_cancellations, 2),
        levels = 0:2,
        labels = c("0", "1", "2+")
      )
    )
}

train <- add_groups(train)
validation <- add_groups(validation)
```

We tested the transformations separately and together, using Model 11 as the starting point.

```
# Model 12: Group special requests
formula12 <- update(
  formula(model11),
  . ~ . - total_of_special_requests + requests_group
)

# Model 13: Group previous cancellations
formula13 <- update(
  formula(model11),
  . ~ . - previous_cancellations + prev_cancel_group
```

```
)

# Model 14: Group both
formula14 <- update(
  formula(model11),
  . ~ . -
    total_of_special_requests -
    previous_cancellations +
    requests_group +
    prev_cancel_group
)
```

We compared these specifications using cross-validation to see whether the extra flexibility improved predictions.

4.8 Cross-validation and final model selection

So far, we had compared models using AIC and validation Brier scores. However, repeatedly testing different specifications on the same validation set risks favouring models that happen to perform well on that sample.

We therefore used **five-fold cross-validation** on the training data. Each model was fitted on four folds and evaluated on the fifth, repeating until every booking had been held out once.

The dataset contains identical records, potentially representing multiple rooms from the same booking. To avoid evaluating a model on copies of records used in fitting, we assigned identical records to the same fold. We also approximately stratified the folds by cancellation outcome.

```
set.seed(123)

# Identify groups of identical bookings
original_cols <- setdiff(names(data), "split")

duplicate_group <- train[, original_cols, drop = FALSE] %>%
  group_by(across(everything())) %>%
  group_indices()

# Assign duplicate groups to folds, stratified by outcome
group_info <- data.frame(
  group_id = duplicate_group,
  is_canceled = train$is_canceled
) %>%
  group_by(group_id) %>%
  summarise(
    is_canceled = first(is_canceled),
    .groups = "drop"
  ) %>%
```

```
  group_by(is_canceled) %>%
  mutate(
    fold = sample(rep(1:5, length.out = n()))
  ) %>%
  ungroup()

cv_fold <- group_info$fold[
  match(duplicate_group, group_info$group_id)
]

table(cv_fold)
```

```
## cv_fold
##     1     2     3     4     5
## 16628 16638 16461 16833 16764
```

We used the same folds for every candidate model and calculated the Brier score from the pooled out-of-fold predictions:

$$\text{CV Brier} = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \hat{p}_i^{(-k(i))}\right)^2,$$

where $\hat{p}_i^{(-k(i))}$ is the prediction from the model fitted without booking i's fold.

Comparing the candidate models

We compared Models 5, 7 and 10–14 using the same cross-validation folds. The full fitting code is retained in our exploratory R Markdown; here we summarise the results without rerunning the models.

```
cv_comparison <- data.frame(
  Model = c(
    "Model 5: Expanded regression",
    "Model 7: Natural cubic splines",
    "Model 10: Hotel-ADR interaction",
    "Model 11: Adding agent",
    "Model 12: Grouped special requests",
    "Model 13: Grouped previous cancellations",
    "Model 14: Both predictors grouped"
  ),
  CV_Brier = c(
    0.1286619,
    0.1279327,
    0.1277105,
    0.1234364,
    0.1223787,
    0.1225725,
    0.1214840
  )
```

```
)

knitr::kable(
  cv_comparison,
  digits = 5,
  caption = "Five-fold cross-validation Brier scores"
)
```

Five-fold cross-validation Brier scores

Model	CV_Brier
Model 5: Expanded regression	0.12866
Model 7: Natural cubic splines	0.12793
Model 10: Hotel-ADR interaction	0.12771
Model 11: Adding agent	0.12344
Model 12: Grouped special requests	0.12238
Model 13: Grouped previous cancellations	0.12257
Model 14: Both predictors grouped	0.12148

Model 14 performed best, with a cross-validation Brier score of 0.12148. Both grouped predictors improved performance individually, and combining them reduced the score further. Since this only added three parameters compared with Model 11, we selected Model 14 as our final specification.

Comparing link functions

We also compared logit, probit and complementary log-log links using the same predictors and cross-validation folds.

```
link_results <- data.frame(
  Link = c("Logit", "Probit", "Complementary log-log"),
  CV_Brier = c(0.1214840, 0.1217005, 0.1223787)
)

knitr::kable(
  link_results,
  digits = 5,
  caption = "Cross-validation comparison of link functions"
)
```

Cross-validation comparison of link functions

Link	CV_Brier
Logit	0.12148
Probit	0.12170
Complementary log-log	0.12238

Logit gave the lowest Brier score, although probit performed almost as well. We therefore kept the logit link.

Fitting the final model

We fitted Model 14 on the full training set and evaluated it on the validation set.

```
model14 <- glm(
  formula14,
  data = train,
  family = binomial(link = "logit"),
  control = glm.control(maxit = 50)
)

stopifnot(model14$converged)

p14 <- as.numeric(
  predict(model14, validation, type = "response")
)

stopifnot(all(is.finite(p14)))

brier14 <- mean((p14 - validation$is_canceled)^2)
```

Model 14 also improved on Model 11 in validation, reducing the Brier score from 0.12609 to 0.12385. Overall, adding predictors such as country and agent produced the largest gains, while splines, interactions and grouped variables offered smaller improvements.

We carried Model 14 forward to examine its predictions in more detail.

5. Checking the final model

5.1 Model interpretation and calibration

We first examined how Model 14's predictions varied with ADR for the two hotels, before checking how closely its predicted probabilities matched the observed cancellation rates.

Interpreting the hotel–ADR interaction

Model 14 includes a hotel-specific ADR spline, allowing the relationship between room price and cancellation risk to differ between hotels. We compared predictions for the same example booking, varying only hotel and ADR over a price range represented in both hotels.

```
# Find agents represented in both hotels
common_agents <- intersect(
  unique(train$agent[train$hotel == "City Hotel"]),
  unique(train$agent[train$hotel == "Resort Hotel"])
)

# Select a reference booking
```

```r
eligible <- which(
  train$hotel == "City Hotel" &
  train$deposit_type == "No Deposit" &
  train$previous_cancellations == 0 &
  train$total_of_special_requests == 0 &
  train$agent %in% common_agents
)

stopifnot(length(eligible) > 0)

reference_i <- eligible[
  which.min(abs(
    train$adr[eligible] - median(train$adr[eligible])
  ))
]

reference_booking <- train[reference_i, , drop = FALSE]

# Restrict ADR to the central range shared by both hotels
adr_by_hotel <- split(train$adr, train$hotel)

adr_min <- max(sapply(
  adr_by_hotel,
  function(x) quantile(x, 0.05, na.rm = TRUE)
))

adr_max <- min(sapply(
  adr_by_hotel,
  function(x) quantile(x, 0.95, na.rm = TRUE)
))

stopifnot(adr_min < adr_max)

adr_grid <- seq(adr_min, adr_max, length.out = 100)

# Hold other booking characteristics fixed
plot_data <- reference_booking[
  rep(1, 2 * length(adr_grid)), ,
  drop = FALSE
]

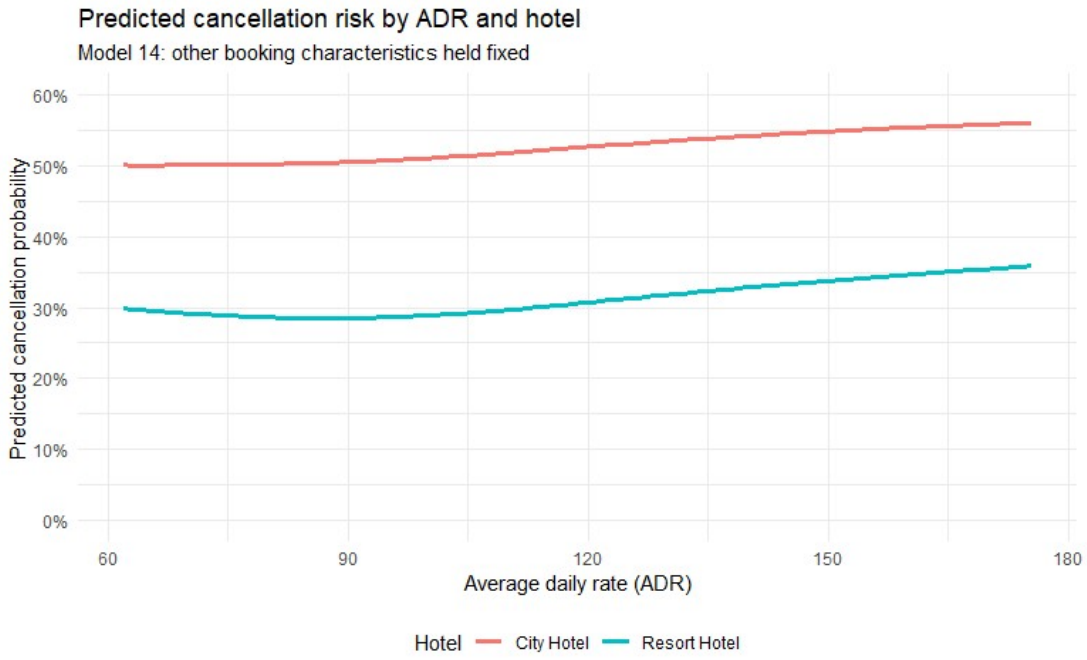
plot_data$adr <- rep(adr_grid, times = 2)

plot_data$hotel <- factor(
  rep(
    c("City Hotel", "Resort Hotel"),
    each = length(adr_grid)
  ),
```

```r
  levels = c("City Hotel", "Resort Hotel")
)

# Predict with Model 14
plot_data$predicted <- as.numeric(
  predict(model14, newdata = plot_data, type = "response")
)

ggplot(
  plot_data,
  aes(x = adr, y = predicted, colour = hotel)
) +
  geom_line(linewidth = 1.2) +
  scale_y_continuous(
    labels = scales::percent,
    breaks = seq(0, 0.6, 0.1)
  ) +
  coord_cartesian(ylim = c(0, 0.6)) +
  labs(
    title = "Predicted cancellation risk by ADR and hotel",
    subtitle = "Model 14: other booking characteristics held fixed",
    x = "Average daily rate (ADR)",
    y = "Predicted cancellation probability",
    colour = "Hotel"
  ) +
  theme_minimal() +
  theme(legend.position = "bottom")
```



For our example booking, cancellation risk increases gradually with ADR at City Hotel, while Resort Hotel shows a slight decrease before increasing at higher prices. City Hotel also has a higher predicted cancellation probability throughout the range. This illustrates the hotel–ADR interaction, although these are conditional predictions rather than evidence that changing prices causes cancellations.

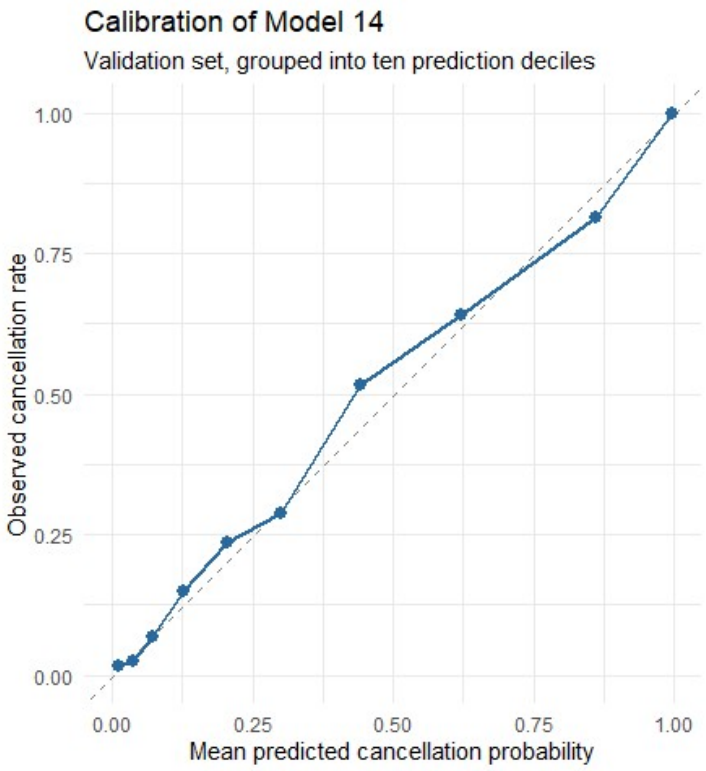
Calibration

We next checked whether the predicted probabilities agreed with observed cancellation rates. We divided the validation predictions into ten equally sized groups and compared the mean predicted probability with the observed cancellation rate in each group.

```
calibration_summary <- validation %>%
  mutate(
    predicted = as.numeric(p14),
    decile = ntile(predicted, 10)
  ) %>%
  group_by(decile) %>%
  summarise(
    mean_predicted = mean(predicted),
    observed_rate = mean(is_canceled),
    n = n(),
    .groups = "drop"
  )

ggplot(
  calibration_summary,
  aes(
    x = mean_predicted,
    y = observed_rate
  )
) +
  geom_abline(
    intercept = 0,
    slope = 1,
    linetype = "dashed",
    colour = "grey50"
  ) +
  geom_line(colour = "#2869A0", linewidth = 1) +
  geom_point(colour = "#2869A0", size = 2.5) +
  coord_equal(
    xlim = c(0, 1),
    ylim = c(0, 1)
  ) +
  labs(
    title = "Calibration of Model 14",
    subtitle = "Validation set, grouped into ten prediction deciles",
    x = "Mean predicted cancellation probability",
    y = "Observed cancellation rate"
```

```
  ) +
  theme_minimal()
```



The predictions are generally close to the dashed line representing perfect calibration, although there are some deviations in the middle of the range.

Calibration intercept and slope

We also fitted a logistic regression of the observed outcomes against the log-odds of the predicted probabilities. Perfect calibration would give an intercept of zero and a slope of one.

```
# Avoid infinite log-odds
p14_clipped <- pmin(
  pmax(as.numeric(p14), 1e-10),
  1 - 1e-10
)

calibration_fit <- glm(
  validation$is_canceled ~ qlogis(p14_clipped),
  family = binomial(link = "logit")
)

calibration_results <- data.frame(
  Measure = c(
    "Calibration intercept",
```

```
    "Calibration slope",
    "Mean predicted probability",
    "Observed cancellation rate"
  ),
  Value = c(
    unname(coef(calibration_fit)[1]),
    unname(coef(calibration_fit)[2]),
    mean(p14),
    mean(validation$is_canceled)
  )
)

knitr::kable(
  calibration_results,
  digits = 4,
  caption = "Calibration statistics for Model 14"
)
```

Calibration statistics for Model 14

Measure	Value
Calibration intercept	0.0393
Calibration slope	0.9609
Mean predicted probability	0.3674
Observed cancellation rate	0.3751

The calibration intercept (0.039) and slope (0.961) are both close to their ideal values, although the slope suggests slightly overconfident predictions. The average predicted cancellation rate was 36.74%, compared with an observed rate of 37.51%, indicating a small overall underestimation.

Overall, Model 14 appears reasonably well calibrated on the validation data. The final test set provides a more independent assessment of its predictive performance.

5.2 Discrimination and classification thresholds

We also assessed how well Model 14 distinguishes cancelled from non-cancelled bookings, using ROC-AUC and average precision alongside Brier score and log-loss.

Validation performance

```
y <- validation$is_canceled
p <- as.numeric(p14)

# Brier score and log-loss
brier <- mean((p - y)^2)

p_clipped <- pmin(pmax(p, 1e-15), 1 - 1e-15)
```

```r
logloss <- -mean(
  y * log(p_clipped) +
  (1 - y) * log(1 - p_clipped)
)

# ROC-AUC
n1 <- sum(y == 1)
n0 <- sum(y == 0)

ranks <- rank(p, ties.method = "average")

auc <- (
  sum(ranks[y == 1]) - n1 * (n1 + 1) / 2
) / (n1 * n0)

# Average precision, accounting for ties
ord <- order(p, decreasing = TRUE)
y_sorted <- y[ord]
p_sorted <- p[ord]

ends <- cumsum(rle(p_sorted)$lengths)
tp <- cumsum(y_sorted)[ends]

precision <- tp / ends
recall <- tp / sum(y_sorted)

ap <- sum(c(recall[1], diff(recall)) * precision)

validation_metrics <- data.frame(
  Metric = c(
    "Brier score",
    "Log-loss",
    "ROC-AUC",
    "Average precision"
  ),
  Value = c(brier, logloss, auc, ap)
)

knitr::kable(
  validation_metrics,
  digits = 4,
  caption = "Validation performance of Model 14"
)
```

Validation performance of Model 14

Metric	Value

Metric	Value
Brier score	0.1239
Log-loss	0.3808
ROC-AUC	0.8961
Average precision	0.8595

Model 14 achieved an ROC-AUC of 0.896 and average precision of 0.860, substantially above the cancellation rate of 0.375. Together with a Brier score of 0.12385, these results suggest good discrimination and reasonably accurate probability predictions.

Choosing a classification threshold

Although our main aim is to predict cancellation probabilities, a hotel might want to flag bookings at higher risk of cancellation. We compared thresholds of 0.3, 0.5 and 0.7 to see how this affects precision and recall.

```
thresholds <- c(0.3, 0.5, 0.7)

threshold_results <- do.call(
  rbind,
  lapply(thresholds, function(t) {

    predicted <- as.integer(p >= t)

    TP <- sum(predicted == 1 & y == 1)
    FP <- sum(predicted == 1 & y == 0)
    FN <- sum(predicted == 0 & y == 1)
    TN <- sum(predicted == 0 & y == 0)

    data.frame(
      Threshold = t,
      Flagged = TP + FP,
      Precision = TP / (TP + FP),
      Recall = TP / (TP + FN),
      Accuracy = (TP + TN) / length(y),
      Missed_Cancellations = FN
    )
  })
)

knitr::kable(
  threshold_results,
  digits = 3,
  caption = "Classification performance at different thresholds"
)
```

Classification performance at different thresholds

Threshold	Flagged	Precision	Recall	Accuracy	Missed_Cancellations
0.3	7717	0.701	0.832	0.804	1094
0.5	5435	0.806	0.674	0.817	2119
0.7	3601	0.897	0.497	0.790	3270

At a threshold of 0.5, the model achieved 81.7% accuracy, with precision of 80.6% and recall of 67.4%. Lowering the threshold to 0.3 increased recall to 83.2%, but precision fell to 70.1%. At 0.7, precision increased to 89.7%, although recall dropped to 49.7%.

This shows the trade-off between identifying more cancellations and incorrectly flagging bookings that would not cancel. We have not chosen an optimal threshold, since this would depend on the hotel's costs and priorities. Keeping the predicted probabilities gives hotels the flexibility to make that decision themselves.

5.3 Subgroup performance and sensitivity to deposit information

We checked whether Model 14 performed similarly across the two hotels and how much its predictions depended on deposit information.

Performance by hotel

```
hotel_performance <- validation %>%
  mutate(predicted = as.numeric(p14)) %>%
  group_by(hotel) %>%
  summarise(
    Bookings = n(),
    Observed_Rate = mean(is_canceled),
    Mean_Predicted = mean(predicted),
    Brier = mean((predicted - is_canceled)^2),
    .groups = "drop"
  )

knitr::kable(
  hotel_performance,
  digits = 4,
  caption = "Validation performance by hotel"
)
```

Validation performance by hotel

hotel	Bookings	Observed_Rate	Mean_Predicted	Brier
City Hotel	11115	0.4202	0.4150	0.1253
Resort Hotel	6216	0.2946	0.2823	0.1212

The Brier scores were similar for City Hotel (0.125) and Resort Hotel (0.121). Predicted cancellation rates were also close to the observed rates, with a slight underestimation for both hotels.

Sensitivity to deposit information

One unusual finding was that approximately 99.7% of Non Refund bookings in the validation set were cancelled. This makes deposit type a strong predictor, but raises questions about how much of our model's performance relies on this relationship.

We investigated this by refitting Model 14 without `deposit_type`, keeping the other predictors unchanged. The full fitting and cross-validation code is retained in our exploratory analysis.

```
deposit_ablation <- data.frame(
  Model = c(
    "Model 14",
    "Model 14 without deposit type"
  ),
  CV_Brier = c(
    0.1214840,
    0.1248508
  ),
  Validation_Brier = c(
    0.1238543,
    0.1288764
  )
)

knitr::kable(
  deposit_ablation,
  digits = 5,
  caption = "Effect of removing deposit type"
)
```

Effect of removing deposit type

Model	CV_Brier	Validation_Brier
Model 14	0.12148	0.12385
Model 14 without deposit type	0.12485	0.12888

Removing deposit type increased both the cross-validation and validation Brier scores, confirming that it provides useful predictive information.

We also compared performance within each deposit category to see where this improvement came from.

```
deposit_subgroups <- data.frame(
  Deposit_Type = c(
```

```
    "No Deposit",
    "Non Refund",
    "Refundable"
  ),
  Bookings = c(15288, 2027, 16),
  Brier_Full = c(
    0.1400743,
    0.0023232,
    0.0220879
  ),
  Brier_Without_Deposit = c(
    0.1404081,
    0.0427691,
    0.0189729
  )
)

knitr::kable(
  deposit_subgroups,
  digits = c(0, 0, 5, 5),
  caption = "Validation Brier scores by deposit type"
)
```

Validation Brier scores by deposit type

Deposit_Type	Bookings	Brier_Full	Brier_Without_Deposit
No Deposit	15288	0.14007	0.14041
Non Refund	2027	0.00232	0.04277
Refundable	16	0.02209	0.01897

Most of the improvement came from Non Refund bookings, where removing deposit type increased the Brier score from 0.0023 to 0.0428. Performance for No Deposit bookings barely changed. The Refundable category contained only 16 bookings, so we cannot draw much from its results.

Limitations

The unusually high cancellation rate among Non Refund bookings is difficult to explain from the data alone. We cannot be sure whether deposit information would always be available and unchanged when making predictions in practice, or whether this relationship would hold for other hotels.

If predictors were updated after the booking was made, our results could be overly optimistic for predicting cancellations in advance. Although the model performed similarly across both hotels, we would need to test it on later bookings before relying on it in practice.

6. Final test-set results and model comparison

We evaluated Model 14 on the 17,903 test bookings, which had not been used during model selection. This provides a more independent assessment of performance, although the test data still comes from the same years as the training data.

6.1 Test-set performance

We applied the same predictor transformations to the test data and calculated the group's evaluation metrics.

```
# Apply the transformations defined in Section 4.7
test_final <- add_groups(test)

p_test14 <- as.numeric(
  predict(model14, test_final, type = "response")
)

y_test <- test_final$is_canceled

stopifnot(
  length(p_test14) == 17903,
  all(is.finite(p_test14)),
  all(p_test14 >= 0 & p_test14 <= 1)
)

# Brier score and log-loss
test_brier <- mean((p_test14 - y_test)^2)

p_safe <- pmin(pmax(p_test14, 1e-15), 1 - 1e-15)

test_logloss <- -mean(
  y_test * log(p_safe) +
  (1 - y_test) * log(1 - p_safe)
)

# ROC-AUC
n1 <- sum(y_test == 1)
n0 <- sum(y_test == 0)

ranks <- rank(p_test14, ties.method = "average")

test_auc <- (
  sum(ranks[y_test == 1]) - n1 * (n1 + 1) / 2
) / (n1 * n0)

# Average precision, accounting for ties
ord <- order(p_test14, decreasing = TRUE)
```

```r
y_sorted <- y_test[ord]
p_sorted <- p_test14[ord]

ends <- cumsum(rle(p_sorted)$lengths)
tp <- cumsum(y_sorted)[ends]

precision <- tp / ends
recall <- tp / sum(y_sorted)

test_ap <- sum(
  c(recall[1], diff(recall)) * precision
)

# Accuracy at a threshold of 0.5
test_accuracy <- mean(
  as.integer(p_test14 >= 0.5) == y_test
)
```

Model 14 achieved a test Brier score of 0.12246, close to its validation score of 0.12385. Its ROC-AUC was 0.8932, with classification accuracy of 82.3% at a threshold of 0.5.

```r
actual <- factor(y_test, levels = 0:1, labels = c("Not cancelled",
"Cancelled"))
predicted <- factor(as.integer(p_test14 >= 0.5),
                    levels = 0:1,
                    labels = c("Not cancelled", "Cancelled"))
confusion <- table(Actual = actual, Predicted = predicted)
knitr::kable(as.data.frame.matrix(confusion),
             caption = "Test confusion matrix (threshold = 0.5)")
```

Test confusion matrix (threshold = 0.5)

	Not cancelled	Cancelled
Not cancelled	10208	1115
Cancelled	2059	4521

The confusion matrix shows that some cancellations are still missed at a threshold of 0.5, highlighting why accuracy alone is not enough to assess the model and why the choice of threshold depends on how the predictions will be used.

6.2 Comparison with Random Forest and XGBoost

We compared Model 14 with the Random Forest and XGBoost models using the same held-out test bookings. The other models' results were taken from the group's shared evaluation.

```r
final_results <- data.frame(
  Metric = c(
    "Brier score",
```

```
    "Log-loss",
    "ROC-AUC",
    "Average precision",
    "Accuracy (0.5)"
  ),
  `Logistic regression` = c(
    test_brier,
    test_logloss,
    test_auc,
    test_ap,
    test_accuracy
  ),
  `Random forest` = c(
    0.1020917,
    0.3157839,
    0.9286631,
    0.8990976,
    0.8510864
  ),
  XGBoost = c(
    0.1014229,
    0.3108823,
    0.9303552,
    0.8999944,
    0.8511423
  ),
  check.names = FALSE
)

knitr::kable(
  final_results,
  digits = 4,
  caption = "Test-set comparison of the three models"
)
```

Test-set comparison of the three models

Metric	Logistic regression	Random forest	XGBoost
Brier score	0.1225	0.1021	0.1014
Log-loss	0.3804	0.3158	0.3109
ROC-AUC	0.8932	0.9287	0.9304
Average precision	0.8594	0.8991	0.9000
Accuracy (0.5)	0.8227	0.8511	0.8511

Both tree-based models performed better than logistic regression across all five metrics. XGBoost had the lowest Brier score (0.10142), followed closely by Random Forest (0.10209), compared with 0.12246 for logistic regression.

This was not entirely unexpected. Although we introduced splines and interactions, logistic regression still required us to specify these relationships manually, whereas the tree-based models can capture more complex patterns automatically.

The difference between Random Forest and XGBoost was small, and the group's paired evaluation found that its 95% confidence interval included zero. A paired comparison including logistic regression would be needed to assess the statistical significance of its performance differences.

Overall, Random Forest and XGBoost gave more accurate predictions on this test set, while logistic regression provided a more transparent model for investigating the relationships between booking characteristics and cancellation risk.

7. Summary and limitations

We developed a logistic regression model to predict hotel booking cancellations, improving its performance through additional predictors, splines, interactions and grouped variables. Our final model achieved a test Brier score of 0.12246, although Random Forest and XGBoost both performed better. Logistic regression was still useful for understanding how different booking characteristics relate to cancellation risk. In practice, hotels could use the predicted probabilities to estimate how many bookings are likely to cancel, helping with room availability and capacity planning, rather than simply classifying individual bookings as cancellations.

There are some important limitations; our model relies partly on the unusually high cancellation rate among non-refundable bookings, which may not hold for other hotels. We also cannot be certain that every predictor would be available and unchanged at the time a prediction was needed. Although logistic regression is relatively interpretable, our final model contains 187 parameters, making some relationships difficult to explain. Finally, our random data split does not tell us how well the model would perform on future bookings. Before using it in practice, we would need to test it on later data and check that its predicted probabilities remain reliable.

The predictions on the test set are in test_preds.csv